Python 接口自动化测试 Day 1 讲义 + 配套练习

Day 1 讲义主题是网络三、数据格式和接口调试基础，HTTP 请求与响应，urllib 模块，JSON 处理，正则表达式，调试工具。

适用对象：入门测试工程师 / 转测试

前置条件：会写测试用例，用过浏览器开发者工具

预计时长：约 3 小时（讲义 1 小时，练习 2 小时）

目录

- 本节学习目标
- 接口测试概念
- HTTP 协议基础知识
  - 请求结构
  - 响应结构
  - 常见状态码
- urllib 模块实战
- JSON 数据处理
- 正则表达式入门
- 调试工具
- 配套练习与答案

Python 接口自动化测试 Day 1 讲义 + 配套练习

一、使用场景

这节课是 Day 1 的“核心工具箱”环节。

主要解决的问题：

- 不会抓包，看不懂请求和响应
- 不会拼接 URL、构造 Header、发送简单请求
- 拿到接口文档，不知道怎么调试
- 接口测试报错，不知道是网络问题还是参数问题
- 写脚本时，不会处理 JSON、Cookie、Token
- 不会用正则从响应中提取关键信息

前置条件：会写测试用例，用过浏览器开发者工具，会写简单 Excel 表格，能看懂 HTTP 请求方法（GET、POST）

预计时长：讲义 1 小时，练习 2 小时，复习 30 分钟

二、本节学习目标

学完本节后，你要能独立完成以下任务：

- 理解接口、请求、响应、HTTP 请求结构、状态码、Header、Cookie、Token、JSON、正则表达式、调试工具的作用
- 使用浏览器开发者工具查看请求和响应
- 使用 urllib 模块发送 GET 请求
- 使用 urllib 模块发送 POST 请求，并传入 JSON 数据
- 使用 urllib 模块设置 Header，并处理 Cookie
- 使用 Fiddler / Charles / Postman 抓包，并能看懂请求方法、URL、Header、Cookie、Body、状态码、响应时间
- 从 JSON 响应中提取关键字段
- 使用正则表达式从文本中提取手机号、验证码、订单号、Token
- 能独立完成 Day 1 配套练习，并能对照答案检查自己的错误

三、接口测试概念：什么是接口测试

3.1 定义

接口测试，就是模拟客户端发起请求，检查服务端返回的结果是否符合预期。

可理解为测试“前后端对接”这一环节是否正确。

- 请求是否发出
- 参数是否正确
- 服务端是否收到
- 业务逻辑是否正确
- 数据是否写入数据库
- 响应格式是否正确
- 字段是否缺失
- 状态码是否合理
- 响应时间是否可接受

3.2 接口测试在测试体系中的位置

通常在单元测试之后、UI 测试之前。

- 单元测试：测试函数、类、模块内部逻辑
- 接口测试：测试服务之间、前端与后端、第三方系统之间的数据交换
- UI 测试：模拟用户点击、输入、页面跳转

接口测试的优点：稳定、执行速度快、不受前端页面改版影响、可以覆盖大量异常场景、可以自动化回归。

3.3 接口测试不能测什么

- 页面显示是否美观
- 按钮是否对齐
- 弹窗文案是否合适
- 用户操作流程是否顺畅

这些属于 UI 测试、人工测试、可用性测试的范围。

3.4 接口测试常见对象

对象	测试重点	UI 测试
注册登录接口	参数校验、错误提示、Token 生成	页面交互体验
列表查询	分页、筛选、排序、空数据	表格渲染
提交表单	必填项、格式校验、重复提交	弹窗提示
文件上传	大小限制、格式限制、失败重试	进度条显示
第三方回调	签名校验、幂等性、超时重试	页面跳转

四、HTTP 协议基础知识

4.1 请求结构

一个完整的请求，可以拆成：

- 请求方法：GET、POST、PUT、DELETE
- 请求 URL：/api/v1/orders?page=1&size=20
- 请求 Header：Content-Type、Cookie、Authorization、User-Agent
- 请求 Body：登录、注册、提交表单、上传文件、调用 JSON 接口

4.2 响应结构

一个完整的响应，通常包含：

- 状态行：HTTP/1.1 200 OK
- 响应 Header：Content-Type、Set-Cookie、Cache-Control、Location
- 响应 Body：HTML、JSON、文件流、错误提示

4.3 常见状态码

状态码	含义	测试关注点
200	请求成功	返回内容是否正确
201	创建成功	是否生成了新数据
204	成功，无内容	删除接口常见
400	参数错误	参数校验是否合理
401	未登录	是否提示登录
403	无权限	权限是否生效
404	接口不存在	路径是否写错
500	服务端错误	是否暴露堆栈信息
502	网关错误	是否重试
504	网关超时	是否设置合理超时

4.4 Cookie、Session、Token、JWT 简单区别

- Cookie：浏览器保存，请求自动带上
- Session：服务端保存，通常配合 Cookie 使用
- Token：客户端保存，手动放入 Header
- JWT：一种常见 Token 格式，内含用户信息，通常有过期时间

五、urllib 模块实战

5.1 发送 GET 请求

```
import urllib.request

response = urllib.request.urlopen("https://example.com/api/ping")
print(response.status)
print(response.read.decode("utf-8"))
```

5.2 发送 POST 表单

```
import urllib.request
import urllib.parse

data = urllib.parse.urlencode({"username": "test", "password": "123456"})
response = urllib.request.urlopen("https://example.com/api/login", data=data.encode("utf-8"))
print(response.status)
print(response.read.decode("utf-8"))
```

5.3 发送 JSON 请求

```
import urllib.request
import urllib.parse
import urllib.error
import time

def send_request(method, url, headers=None, data=None, timeout=10):
    if headers is None:
        headers = {}
    if data is None:
        data = b""
    else:
        if isinstance(data, str):
            data = data.encode("utf-8")
    req = urllib.request.Request(url, data=data, headers=headers, method=method)
    start = time.time()
    try:
        response = urllib.request.urlopen(req, timeout=timeout)
        status = response.status
        body = response.read.decode("utf-8")
        except_type = None
        except_msg = None
        return status, body, except_type, except_msg
    except urllib.error.HTTPError as e:
        status = e.code
        body = e.read.decode("utf-8")
        except_type = "HTTPError"
        except_msg = str(e)
        return status, body, except_type, except_msg
    except urllib.error.URLError as e:
        status = None
        body = ""
        except_type = "URLError"
        except_msg = str(e.reason)
        return status, body, except_type, except_msg
    except Exception as e:
        status = None
        body = ""
        except_type = "Exception"
        except_msg = str(e)
        return status, body, except_type, except_msg
    finally:
        elapsed = time.time - start
        print(elapsed)
```

5.4 封装成简单函数

```
import urllib.request
import urllib.parse
import urllib.error
import time

def send_request(method, url, headers=None, data=None, timeout=10):
    if headers is None:
        headers = {}
    if data is None:
        data = b""
    else:
        if isinstance(data, str):
            data = data.encode("utf-8")
    req = urllib.request.Request(url, data=data, headers=headers, method=method)
    start = time.time()
    try:
        response = urllib.request.urlopen(req, timeout=timeout)
        status = response.status
        body = response.read.decode("utf-8")
        except_type = None
        except_msg = None
        return status, body, except_type, except_msg
    except urllib.error.HTTPError as e:
        status = e.code
        body = e.read.decode("utf-8")
        except_type = "HTTPError"
        except_msg = str(e)
        return status, body, except_type, except_msg
    except urllib.error.URLError as e:
        status = None
        body = ""
        except_type = "URLError"
        except_msg = str(e.reason)
        return status, body, except_type, except_msg
    except Exception as e:
        status = None
        body = ""
        except_type = "Exception"
        except_msg = str(e)
        return status, body, except_type, except_msg
    finally:
        elapsed = time.time - start
        print(elapsed)
```

六、JSON 处理

6.1 JSON 字符串转 Python 对象

```
import urllib.request
import urllib.parse
import urllib.error
import time
import json

def send_request(method, url, headers=None, data=None, timeout=10):
    if headers is None:
        headers = {}
    if data is None:
        data = b""
    else:
        if isinstance(data, str):
            data = data.encode("utf-8")
    req = urllib.request.Request(url, data=data, headers=headers, method=method)
    start = time.time()
    try:
        response = urllib.request.urlopen(req, timeout=timeout)
        status = response.status
        body = response.read.decode("utf-8")
        except_type = None
        except_msg = None
        return status, body, except_type, except_msg
    except urllib.error.HTTPError as e:
        status = e.code
        body = e.read.decode("utf-8")
        except_type = "HTTPError"
        except_msg = str(e)
        return status, body, except_type, except_msg
    except urllib.error.URLError as e:
        status = None
        body = ""
        except_type = "URLError"
        except_msg = str(e.reason)
        return status, body, except_type, except_msg
    except Exception as e:
        status = None
        body = ""
        except_type = "Exception"
        except_msg = str(e)
        return status, body, except_type, except_msg
    finally:
        elapsed = time.time - start
        print(elapsed)
```

6.2 Python 对象转 JSON 字符串

```
import urllib.request
import urllib.parse
import urllib.error
import time
import json

def send_request(method, url, headers=None, data=None, timeout=10):
    if headers is None:
        headers = {}
    if data is None:
        data = b""
    else:
        if isinstance(data, str):
            data = data.encode("utf-8")
    req = urllib.request.Request(url, data=data, headers=headers, method=method)
    start = time.time()
    try:
        response = urllib.request.urlopen(req, timeout=timeout)
        status = response.status
        body = response.read.decode("utf-8")
        except_type = None
        except_msg = None
        return status, body, except_type, except_msg
    except urllib.error.HTTPError as e:
        status = e.code
        body = e.read.decode("utf-8")
        except_type = "HTTPError"
        except_msg = str(e)
        return status, body, except_type, except_msg
    except urllib.error.URLError as e:
        status = None
        body = ""
        except_type = "URLError"
        except_msg = str(e.reason)
        return status, body, except_type, except_msg
    except Exception as e:
        status = None
        body = ""
        except_type = "Exception"
        except_msg = str(e)
        return status, body, except_type, except_msg
    finally:
        elapsed = time.time - start
        print(elapsed)
```

七、正则表达式入门

7.1 提取手机号

```
import urllib.request
import urllib.parse
import urllib.error
import time
import re

def send_request(method, url, headers=None, data=None, timeout=10):
    if headers is None:
        headers = {}
    if data is None:
        data = b""
    else:
        if isinstance(data, str):
            data = data.encode("utf-8")
    req = urllib.request.Request(url, data=data, headers=headers, method=method)
    start = time.time()
    try:
        response = urllib.request.urlopen(req, timeout=timeout)
        status = response.status
        body = response.read.decode("utf-8")
        except_type = None
        except_msg = None
        return status, body, except_type, except_msg
    except urllib.error.HTTPError as e:
        status = e.code
        body = e.read.decode("utf-8")
        except_type = "HTTPError"
        except_msg = str(e)
        return status, body, except_type, except_msg
    except urllib.error.URLError as e:
        status = None
        body = ""
        except_type = "URLError"
        except_msg = str(e.reason)
        return status, body, except_type, except_msg
    except Exception as e:
        status = None
        body = ""
        except_type = "Exception"
        except_msg = str(e)
        return status, body, except_type, except_msg
    finally:
        elapsed = time.time - start
        print(elapsed)
```

7.2 提取验证码

```
import urllib.request
import urllib.parse
import urllib.error
import time
import re

def send_request(method, url, headers=None, data=None, timeout=10):
    if headers is None:
        headers = {}
    if data is None:
        data = b""
    else:
        if isinstance(data, str):
            data = data.encode("utf-8")
    req = urllib.request.Request(url, data=data, headers=headers, method=method)
    start = time.time()
    try:
        response = urllib.request.urlopen(req, timeout=timeout)
        status = response.status
        body = response.read.decode("utf-8")
        except_type = None
        except_msg = None
        return status, body, except_type, except_msg
    except urllib.error.HTTPError as e:
        status = e.code
        body = e.read.decode("utf-8")
        except_type = "HTTPError"
        except_msg = str(e)
        return status, body, except_type, except_msg
    except urllib.error.URLError as e:
        status = None
        body = ""
        except_type = "URLError"
        except_msg = str(e.reason)
        return status, body, except_type, except_msg
    except Exception as e:
        status = None
        body = ""
        except_type = "Exception"
        except_msg = str(e)
        return status, body, except_type, except_msg
    finally:
        elapsed = time.time - start
        print(elapsed)
```

7.3 提取订单号

```
import urllib.request
import urllib.parse
import urllib.error
import time
import re

def send_request(method, url, headers=None, data=None, timeout=10):
    if headers is None:
        headers = {}
    if data is None:
        data = b""
    else:
        if isinstance(data, str):
            data = data.encode("utf-8")
    req = urllib.request.Request(url, data=data, headers=headers, method=method)
    start = time.time()
    try:
        response = urllib.request.urlopen(req, timeout=timeout)
        status = response.status
        body = response.read.decode("utf-8")
        except_type = None
        except_msg = None
        return status, body, except_type, except_msg
    except urllib.error.HTTPError as e:
        status = e.code
        body = e.read.decode("utf-8")
        except_type = "HTTPError"
        except_msg = str(e)
        return status, body, except_type, except_msg
    except urllib.error.URLError as e:
        status = None
        body = ""
        except_type = "URLError"
        except_msg = str(e.reason)
        return status, body, except_type, except_msg
    except Exception as e:
        status = None
        body = ""
        except_type = "Exception"
        except_msg = str(e)
        return status, body, except_type, except_msg
    finally:
        elapsed = time.time - start
        print(elapsed)
```

八、调试工具

8.1 浏览器开发者工具

- 看 Network 面板
- 看 Request Headers
- 看 Response Headers
- 看 Payload
- 看 Response
- 看 Cookies
- 看 Timing

8.2 Fiddler / Charles / Postman

- 抓包
- 改请求
- 重发请求
- 看响应
- 保存接口

九、配套练习

练习 1：发送 GET 请求并解析 JSON

- 调用 `https://example.com/api/ping`
- 打印 status
- 打印 body
- 尝试解析 JSON
- 提取 `data.version`

练习 2：发送 POST 表单登录请求

- 调用 `https://example.com/api/login`
- 传入 `username=test&password=123456`
- 打印 status
- 打印 body
- 尝试提取 `data.token`

练习 3：发送 JSON 请求并处理异常

- 调用 `https://example.com/api/echo`
- 传入 `{"name": "test", "age": 18}`
- 打印 status
- 打印 body
- 捕获 HTTPError
- 捕获 URLError
- 捕获 Exception
- 打印 except_type
- 打印 except_msg
- 打印 elapsed

练习 4：从文本中提取手机号、验证码、订单号

- 给一段聊天记录
- 提取 11 位手机号
- 提取 6 位数字验证码
- 提取 `ORD` 开头的订单号

练习 5：封装一个简单的接口测试函数

- 输入：method、url、headers、data、timeout
- 输出：status、body、elapsed、except_type、except_msg
- 要求：异常不崩溃、结果可打印、字段可扩展

十、常见错误

- 只会点按钮，不会看请求和响应
- 不会拼 URL 参数
- 不会加 Header
- 不会处理 Cookie / Token
- 不会解析 JSON
- 不会用正则从文本中提取关键字段
- 不会区分 401 / 403 / 404 / 500
- 不会区分 HTTPError / URLError / Exception
- 不会记录 except_type / except_msg / elapsed
- 不会把接口问题和页面问题混淆

十一、下节课预告

下节课：Day 2 接口测试进阶

- 参数化
- 断言
- 登录态
- 多接口联调
- 简单测试报告

十二、常见问题

Q1：urllib 和 requests 哪个好？

先学 urllib 更容易理解底层。

requests 更适合实际项目，语法更简洁。

Q2：为什么要自己写 send_request？

- 统一异常处理
- 统一超时
- 统一记录耗时
- 统一打印 except_type / except_msg
- 后面写脚本、批量测试、生成报告时，结构更清晰

Q3：Cookie 和 Token 有什么区别？

- Cookie：浏览器自动保存，请求自动带上
- Session：服务端保存，通常配合 Cookie 使用
- Token：客户端保存，手动放入 Header
- JWT：一种常见 Token 格式，内含用户信息，通常有过期时间

Q4：为什么要区分 except_type？

- HTTPError：服务端有响应，但状态码不是 2xx
- URLError：连不上服务器、DNS 失败、超时
- Exception：代码写错、解析失败、字段缺失

区别越清晰，排查越快。

十三、结语

Day 1 的目标不是学完 HTTP 全部内容，而是先建立：

- 能发请求
- 能看响应
- 能拼 URL
- 能处理 Header
- 能处理 Cookie / Token
- 能解 JSON
- 能用正则提取关键字段
- 能用调试工具抓包
- 能把这些东西封装成可复用函数

这一步做完，后面写脚本、做接口测试、做自动化，就不会一上来就卡在基础工具上。

7.1 数据结构题

- 数组和链表的区别，各种应用场景？
- 二叉树遍历方式
- 二叉搜索树
- 哈希表冲突
- 堆的应用场景
- 图的存储方式
- 图广度搜索
- 拓扑排序算法
- 最短路径算法
- 最小生成树

7.2 分治算法题

- 分治算法的适用条件
- 最大子数组问题
- 逆序对问题
- 快速幂
- 矩阵快速幂
- 分治算法的时间复杂度

7.3 动态规划题

- 动态规划的三大特征
- 背包问题
- 最长公共子序列
- 最长递增子序列
- 编辑距离
- 状态压缩DP
- 区间DP
- 树形DP
- 动态规划的时间复杂度

7.4 贪心算法

- 贪心算法的适用条件
- 区间调度问题
- 哈夫曼编码
- 最小生成树
- 最小生成树算法
- 最小生成树的时间复杂度

7.5 搜索题

- 广度搜索
- 深度搜索
- 迭代加深
- 双向搜索
- 剪枝
- 状态压缩
- 搜索的时间复杂度

7.6 字符串

- 字符串匹配
- KMP
- 字典树
- 哈希匹配
- 字符串的时间复杂度

7.7 数学

- 质数
- 欧拉筛
- 最大公约数
- 扩展欧几里得
- 组合数
- 卡特兰数
- 容斥原理
- 概率期望
- 矩阵快速幂

7.8 工具题

- 位运算技巧
- 二进制
- 哈希
- 堆
- 栈
- 队列
- 单调栈
- 单调队列
- 双指针
- 滑动窗口
- 差分
- 前缀和
- 二分
- 三分
- 倍增
- 离散化
- 分治
- 递归
- 回溯
- 剪枝
- 状态压缩
- 时间复杂度
- 空间复杂度

7.9 工程题

- 单元测试
- 日志
- 性能分析
- 内存泄漏
- 线程安全
- 锁
- 死锁
- 缓存
- 消息队列
- 分布式锁
- 幂等
- 限流
- 熔断
- 降级
- 灰度发布
- 回滚
- 监控
- 告警
- 日志分析
- 性能优化

7.10 设计题

- 设计一个缓存
- 设计一个消息队列
- 设计一个搜索引擎
- 设计一个推荐系统
- 设计一个短链接系统
- 设计一个分布式锁
- 设计一个限流器
- 设计一个日志系统
- 设计一个监控系统
- 设计一个配置中心
- 设计一个文件上传系统
- 设计一个聊天系统
- 设计一个排行榜
- 设计一个广告系统
- 设计一个搜索自动补全系统
- 设计一个分布式计数器
- 设计一个延时队列
- 设计一个定时任务系统
- 设计一个数据同步系统
- 设计一个多级缓存系统
- 设计一个高并发秒杀系统
- 设计一个短链接系统

- 你能讲清楚接口测试的基本概念
- 你能看懂接口文档里的关键字段
- 你能解释认证、参数、状态码的核心概念

实操完成

- 你已经用工具手工调通接口
- 你已经记录接口信息
- 你已经列出测试点

如果只完成理论，没有完成实操，不算真正学完。

---

27. 明天衔接

Day 2 会正式进入 Python 实战，重点包括：

- 安装和使用 `requests`
- 用 Python 发送 `GET/POST`
- 处理 `params/data/json`
- 解析状态码和响应体
- 登录并提取 token

因此今天最重要的准备不是继续背概念，而是确保你手里已经有可调试的接口样本。